Animus Core — Benchmark Datasheet

Architecture specifications and measured performance for the C++ telemetry engine

> **Audience:** Lead Quant Developers, Systems Architects, and C++ Performance Engineers.
> Every number in this document is reproducible from source. Sources and exact commands are cited inline rather than asserted — run them yourself before you trust them.

1. Architecture Highlights

- **Zero-copy C-ABI boundary.** The Python SDK talks to the native engine (`AnimusNative.dll` / `libanimus_native.so`) through direct buffer pointers via `ctypes` , not serialized IPC. Batched ingestion (`animus_record_events_batch`) passes a raw byte buffer once per batch rather than marshalling one object per event.
- **Lock-free MPMC ring buffer.** `animus::LockFreeRingBuffer<T>` implements the Vyukov multi-producer/multi-consumer algorithm — the same ring `EngineImpl` 's telemetry path uses internally. Verified correct under real contention: an 8-producer-thread run pushing 1,600,000 records drains back out exactly once per push, with the benchmark harness hard-failing on any mismatch rather than assuming correctness.
- **Cache-line-aware layout.** Hot counters and ring slots are padded to cache-line boundaries to eliminate false sharing between contending threads. Measured impact: **4.55x** throughput improvement from padding alone, on an otherwise-identical two-thread contention benchmark (see §2).
- **Shared-memory IPC option.** `SharedMemorySegment` (POSIX `shm_open` /Windows equivalent) exposes a cross-process ring for deployments that need to fan telemetry out to a separate consumer process without a socket or broker in the loop.
- **Optional hardware-locked feature gating.** CPU core pinning and similar tail-latency tuning knobs are gated by an offline, RSA-2048-verified license — fail-closed by design, with zero effect on the core ingestion path when unlicensed.

2. Latency & Throughput Profile

Methodology note: the figures below come from two distinct measurement layers of the same engine — read the "Layer" implied by each heading before comparing rows. Native, single-threaded decision-loop latency and Python-SDK batched-ingestion throughput answer different engineering questions; this project's benchmark culture treats conflating them as a methodology error, not a rounding choice.

Native C++ decision-loop latency — tick-to-trade

Source: `benchmarks/BENCHMARK_REPORT.md` §1, generated by `AnimusCore_v1/animus_benchmark_suite.cpp` . Reproduce with `python benchmarks/generate_benchmark_report.py` .

Single-threaded, sequential ingest → poll → execute loop (`MarketDataFeed` tick in, `ExecutionClient::submit()` out), 500,000 iterations, pinned to core 0.

Percentile	Latency
p50 (median)	**100 ns**
p99	100 ns
p99.9	**200 ns**

Values are quantized to whole hundreds of nanoseconds by this machine's `steady_clock` resolution (Windows: QueryPerformanceCounter-backed) — true per-call latency may be finer than the clock can distinguish.

Ring buffer throughput — 8-producer contention

Source: `benchmarks/BENCHMARK_REPORT.md` *§2.*

8 concurrent producer `std::thread`s, real OS scheduling across real cores, each pushing 200,000 records (1,600,000 total) onto a pre-sized ring, all contending on the same compare-exchange retry loop.

Metric	Result
Aggregate push throughput	**7,216,498 pushes/sec**
Per-push p50 latency	600 ns
Per-push p99 latency	5,200 ns

This measures producer-side contention only — no concurrent consumer drains the ring during the timed window.

Python SDK batched-ingestion throughput

Source: `AnimusCore_v1/BENCHMARKS.md` *Phase 13 (* `benchmarks/fintech_tail_latency.py` *), representative run out of 5, timed call-by-call at nanosecond resolution across the real* `animus_record_events_batch` *C-ABI call.*

Metric	Representative run	Range across 5 runs
Throughput	**6,891,485 events/sec**	6,615,997 – 6,891,485 ev/s
p50	13.70 µs	13.70 – 14.40 µs
p99	27.60 µs	24.80 – 28.40 µs

This layer includes the Python-side call overhead the native benchmark above does not — it is the honest number for "what does my Python caller actually see."

Cache-line padding — false-sharing A/B

Source: `benchmarks/BENCHMARK_REPORT.md` *§4.*

Layout	Result
Unpadded (false-sharing)	Baseline
Cache-line padded	**4.55x** combined ops/sec

Cross-core SPSC dispatch latency — C++23 telemetry harness

Source: `benchmarks/telemetry_benchmark.cpp` *, representative run out of 3. Reproduce with:* `cl /std:c++latest /EHsc /O2 /DNDEBUG /Fe:telemetry_benchmark.exe benchmarks\telemetry_benchmark.cpp && telemetry_benchmark.exe` *(MSVC) or* `g++ -std=c++23 -O3 -DNDEBUG -pthread -o telemetry_benchmark benchmarks/telemetry_benchmark.cpp -lstdc++exp && ./telemetry_benchmark` *(GCC/Clang+libstdc++).*

One producer thread and one consumer thread, pinned to separate physical cores, handing off cache-line-aligned (64-byte) `TelemetryEvent`s through a lock-free SPSC ring. Timestamps are lfence-serialized RDTSC reads, calibrated against `std::chrono::steady_clock` — giving sub-100ns resolution the QueryPerformanceCounter-quantized figures elsewhere in this document cannot show. Two separate phases: a depth-1 latency phase (1,000,000 events; the producer waits for the consumer's receipt ack before dispatching the next one, so a sample is real transport cost, not queueing backlog) and an unthrottled throughput phase (9,000,000 events, maximum sustained rate).

Latency (depth-1 phase, producer→consumer, cross-core):

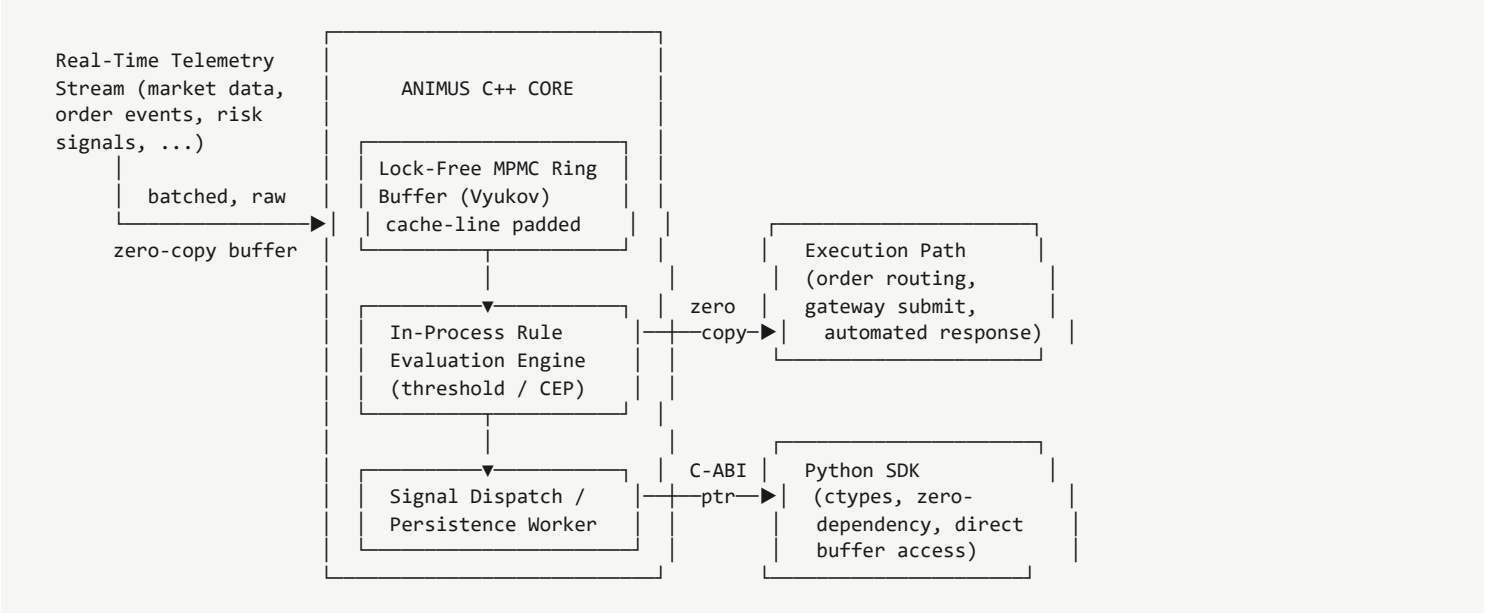
Percentile	Latency	Range across 3 runs
min	45.5 ns	39.7 – 45.5 ns
p50 (median)	**53.3 ns**	52.5 – 53.3 ns
p90	57.9 ns	56.6 – 57.9 ns
p99	**64.9 ns**	62.4 – 64.9 ns
p99.9	111.6 ns	88.5 – 113.7 ns

Throughput (unthrottled flood phase):

Metric	Representative run	Range across 3 runs
Sustained throughput	**47.306 M msgs/sec**	47.306 – 49.497 M msgs/sec

SPSC (one producer, one consumer, zero compare-exchange contention) with a consumer actively draining concurrently — not directly comparable to the 8-producer MPMC contention figure above or the single-threaded, no-cross-core-hop figures elsewhere in this document. Max latency runs into the hundreds of microseconds across all 3 runs, reflecting OS scheduling noise on the pinned cores rather than the transport itself; p50/p90/p99 are the steady-state figures.

3. System Diagram



Everything inside the Animus C++ Core box runs in-process on the ingesting thread — no network hop, no message broker, no serialization step between ring buffer and rule evaluation. The Python SDK and Execution Path are two independent consumers of the same zero-copy boundary, not sequential stages.

4. Client Integration Quickstart

Zero third-party Python dependencies — `animus/` imports only `ctypes`, `threading`, and `multiprocessing.shared_memory` from the standard library.

```
# 1. Install the SDK in editable mode (from a full source checkout)
pip install -e .

# 2. Run the reference pilot integration against the real C-ABI
python Pilot_Kit/animus_integration_example.py [path/to/your_pilot.lic]
```

```
# Minimal ingestion + rule evaluation, using the same interop layer
# every script in this repository uses (animus/bindings.py):
from animus.bindings import AnimusBindings

engine = AnimusBindings()
engine.init(ring_capacity=4096)

# Register a threshold rule -- native, in-process rule evaluation,
# zero-copy, on the same worker thread as ingestion.
engine.add_rule(event_id=1, comparator=">", threshold=2000, severity="HIGH", action="ALERT")

# Batched ingestion -- one C-ABI call per batch, not per event.
# This is the call the throughput/latency numbers in §2 measure directly.
engine.record_events_batch(events)

# Drain any threat signals the rule above matched.
signals = engine.poll_signals()
```

For the full API surface — CEP rule chaining, mTLS-secured multi-tenant transport, distributed Raft-lite clustering, market-data feed adapters, shared-memory IPC — see `AnimusCore_v1/QUICKSTART.md`.

All figures above are drawn directly from this repository's own benchmark suite and are re-runnable — see the "Source" line under each table. Report a discrepancy on your own hardware to your Animus Core contact; per-hardware, per-run variance is expected and documented throughout `AnimusCore_v1/BENCHMARKS.md`.